1. INTRODUCTION

To recap, this project addresses the global shortage of qualified teachers. UNESCO estimates roughly 44 million additional primary and secondary teachers are needed worldwide by 2030 to meet universal education goals, a shortfall felt most acutely in low-income and rural settings. This report details progress made since the April submission toward an AI-driven, facilitator-based education platform addressing that gap.

2. PROJECT PLANNING & TASK ALLOCATION

Since the April submission, May and I revisited the problem statement and narrowed it further, splitting "reducing dependency on expert teachers" into two parts pursued in parallel, both converging on the same underlying goal of making education more accessible and consistent.

Part 1: The Platform. I took responsibility for the facilitator- and admin-facing system itself, covering system architecture, the guided lesson-delivery experience, and the surrounding build, alongside requirements elicitation, interviews, and ethics approval.

Part 2: Student Learning Insights. May took responsibility for developing a system to evaluate student behaviour, skills, and abilities through an online quiz, tracking metrics synthesised from research and turning student data into insights that help a facilitator teach more effectively. This was originally scoped more broadly around analysing student behaviour, then narrowed to focus specifically on facilitator-facing insight, drawing on the psychological and pedagogical research side of the project.

Working in parallel, both parts have progressed substantially. To deepen our understanding of the problem, we also pursued structured interviews and filed a UoA low-risk ethics approval application, currently awaiting sign-off (Section 3).

3. ETHICS APPROVAL

Because the project involves interviews with human participants, approval from the University of Auckland Human Participants Ethics Committee (UAHPEC) was required before any data collection could begin. Given the project timeline, the low-risk/expedited pathway was chosen as the best balance between gathering real primary data and staying on schedule.

Using our user stories as a guide, I prepared and filed the ethics application, Participant Information Sheet, and consent form. UAHPEC is currently reviewing our application, and we are awaiting approval.

4. TARGET AUDIENCE & USER RESEARCH APPROACH

Interviewee profiles were identified and prioritised directly from the platform's target users and the gaps identified in the April literature review. Highest priority went to the platform's core users and the people who manage them day to day: non-expert facilitators and teaching assistants (the platform's core user), experienced classroom teachers, and academic coordinators or school principals, particularly those at Dolphins Group of Schools in India, an affordable-school operator that experiences the exact problem this project targets, where experienced teachers migrate to higher-paying schools and leave less experienced staff behind. Medium priority was given to prospective education entrepreneurs, homeschooling parents in New Zealand, and adults reflecting on their own K-12 schooling, each offering a different angle on the same problem.

Across all profiles, the interviews are designed around three goals: validating the platform's core approach, understanding pain points in how lessons are currently delivered, and surfacing what facilitators actually need to teach effectively without subject expertise.

I will be using thematic analysis to review interview responses, which will include going through written notes across participants to pick out recurring patterns, common pain points, and themes, rather than treating each response in isolation. Trends and common items surfaced this way will guide what comes next: which design

assumptions hold up, what facilitators actually need, and where the platform's current approach should adjust. Keeping this process systematic also keeps it reproducible, so the same themes could be identified independently from the same notes.

5. SYSTEM DESIGN

As the platform side of the project (Section 2, Part 1), the system follows a content hierarchy of Organisation → Class → Subject → Lesson → Block, mirroring how a facilitator naturally organises teaching: by year level, then subject, then individual lessons, matching the mental model of someone responsible for a consistent group of students building toward a shared understanding, rather than an abstract content taxonomy.

This shapes the platform's two server-enforced roles: Administrators create and organise lesson content, while Facilitators deliver it through a guided, step-by-step interface but cannot author it. The split follows directly from the problem being solved: facilitators are assumed not to have subject expertise, so content authoring stays with Administrators, who bring the pedagogical judgement to design a lesson well, while Facilitators execute that design faithfully rather than making instructional calls on the fly. Lessons are accordingly built as an ordered sequence of typed blocks (video, workbook, activity, worksheet, reading, quiz, discussion, break, note) rather than one monolithic format, so lesson structure itself carries guidance a facilitator would otherwise need subject expertise to supply.

Architecturally, Next.js 14's App Router pairs Server Components for data reads with Server Actions for all mutations, removing the need for a separate API layer and letting middleware enforce route protection and session checks before any request reaches application code (Figure 1). Supabase was chosen to cover authentication, database, and file storage from a single vendor, reducing integration surface to one API rather than three. Two decisions worth noting: Prisma was deliberately pinned below v7 rather than migrating the schema, after v7 removed syntax the schema depended on (a stability-over-immediacy trade-off), and Resend was chosen over a mocked email flow specifically so invite emails would be genuinely testable, with a graceful fallback (the invite link shown directly in the admin UI) when no API key is configured, so the approval flow works without live email infrastructure.

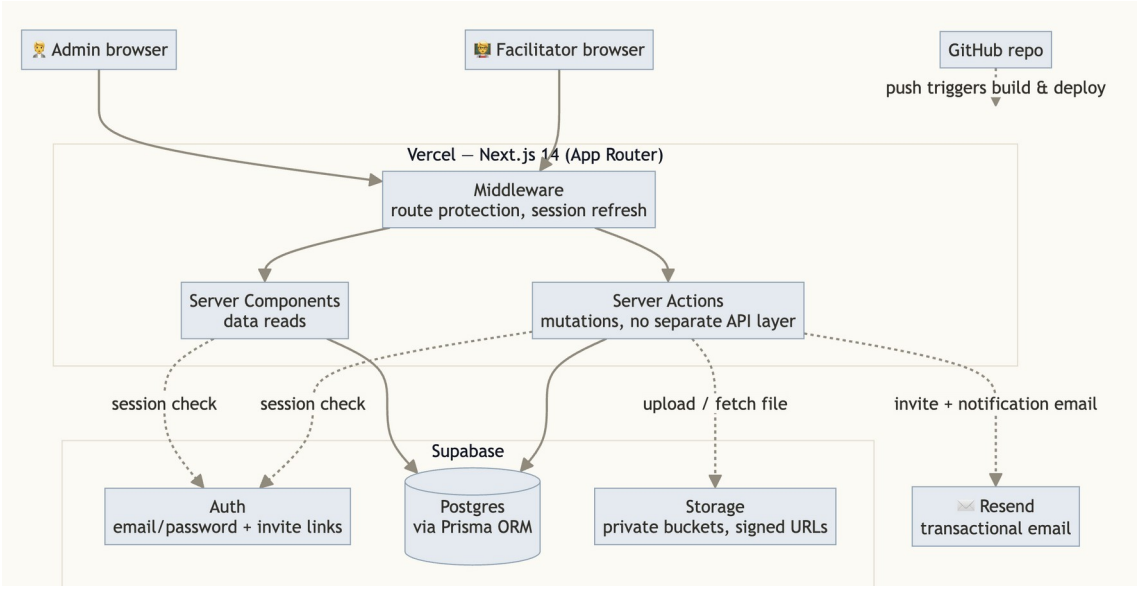


Figure 1: System architecture. Next.js middleware routes requests to Server Components and Server Actions, which read and write Supabase (Auth, Postgres via Prisma, Storage) and trigger Resend for email; GitHub pushes trigger a Vercel build and deploy.

6. AI-ASSISTED DEVELOPMENT APPROACH

AI tooling was used deliberately throughout development, with different tools trialled for different kinds of work before settling on a single pipeline that consistently produced the best results. Antigravity IDE, ChatGPT

Codex, and Claude Code were trialled for coding; Omni Flash, Google Flow, Nano Banana, and Imagen for image and video generation; and ChatGPT 5.5 & 5.6 Sol, Gemini 3, and Claude Sonnet 5 for idea refinement, strategy planning, and requirements documenting.

The pipeline that worked best begins by giving ChatGPT relevant project context and using voice mode to talk through ideas and decisions back and forth, converging on an initial specification document covering the problem definition, brief, and goals, alongside how the system should work, what each page needs, and how the site should look. This document is taken to Claude for thorough refinement, then to Claude Code (running Opus 4.8) for a further pass focused specifically on coding feasibility, resulting in a single project brief covering what is to be built, what each page requires, and what tools will be used.

From that brief, a session-by-session build plan is generated, breaking the work into small pieces that can be thought of as AI coding sprints using agile development. This becomes an iterative process using Claude Code's Opus 4.8 model, with maximum reasoning and extended thinking, to build each session and then test it in real time manually before moving to the next section. The specification itself continues to be refined as code is written, rather than treated as fixed upfront. Every three to four iterations, a full code review is run using Claude Fable 5, chosen specifically for code review and bug-finding. Connectors, MCPs, and custom UI/UX skills keep generated work consistent with the design system throughout, and context carries between sessions via structured handoff documents, one of which (a full project-status handoff) was used directly as source material for this report.

7. IMPLEMENTATION: WHAT WAS BUILT & KEY DESIGN DECISIONS

User onboarding and validation. The platform hands real responsibility to people without subject expertise, and that is why who gets access and what they can do needed deliberate control rather than convenience. Hiding actions in the interface alone would not stop a facilitator from reaching an admin action by guessing a URL, so roles are enforced server-side at the request level instead. Letting anyone create their own facilitator account would mean no real control over who ends up delivering lessons, which is why accounts follow a request-first, invite-later model: when a user signs up, a request is sent to the admin, and only once the admin approves it can the user access any content. The signup form also asks the user which classes they would like access to, so that when an admin approves the account, access to those requested classes is granted in the same step, avoiding "idle accounts" with no class to actually teach. This also makes onboarding quicker, since verification and class allocation happen as one action rather than two separate steps.

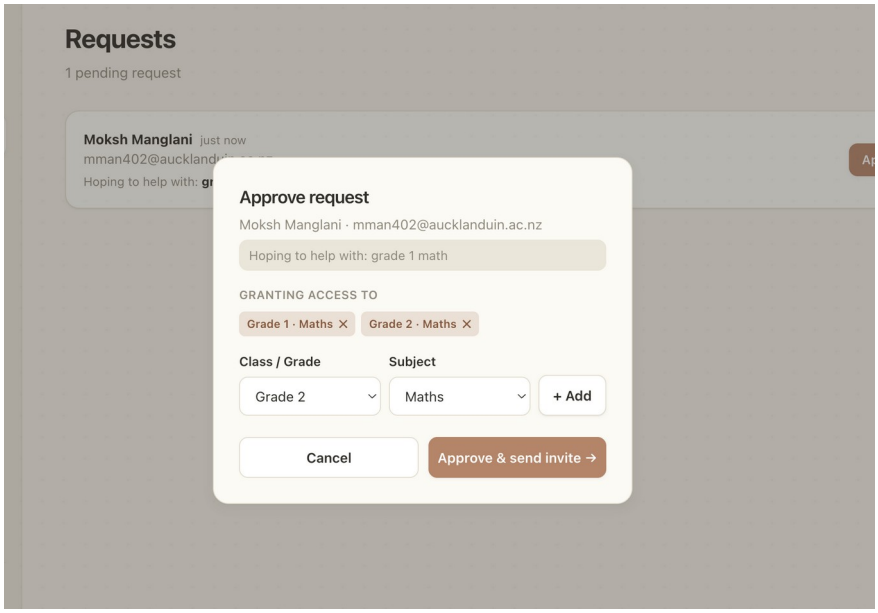


Figure 2: Approving a signup request. Class and subject access is granted in the same step as approval, so a facilitator is never left with an approved account and no class to teach.

A real authoring system. Admins carry the pedagogical expertise, and in order for them to create engaging lessons, we needed to provide an interface that is flexible and modular rather than a generic upload box. That is why we created the concept of Blocks, individual modular components that can be used depending on the task required. Lessons are built with a drag-and-drop block builder across nine block types, each with fields suited to it: a video block takes a URL and duration, a worksheet block takes an uploaded file, and so on. Every lesson sits as a draft on the admin panel, and once it is finalised and tested, it can be published to the relevant class under the chosen subject. Publishing is not final either, since a lesson might still need fixing or pulling from view later, which is why admins can hide or edit it even after it has gone live.

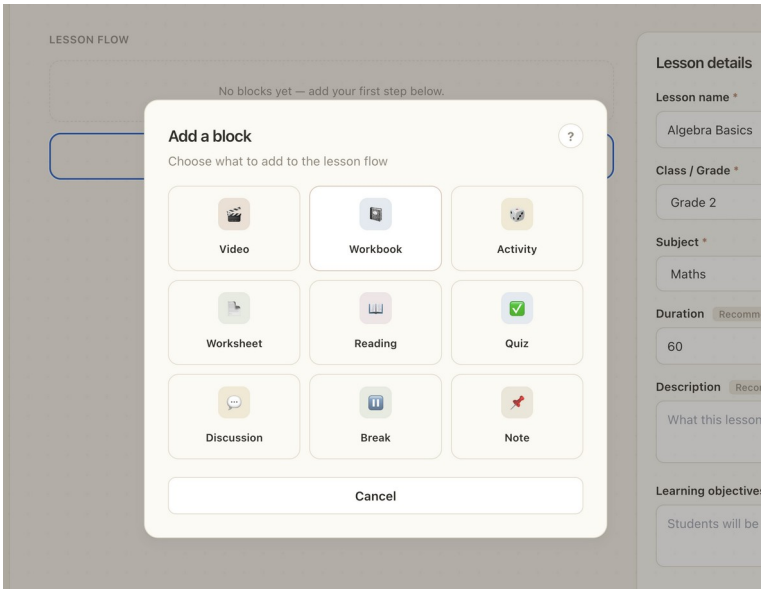
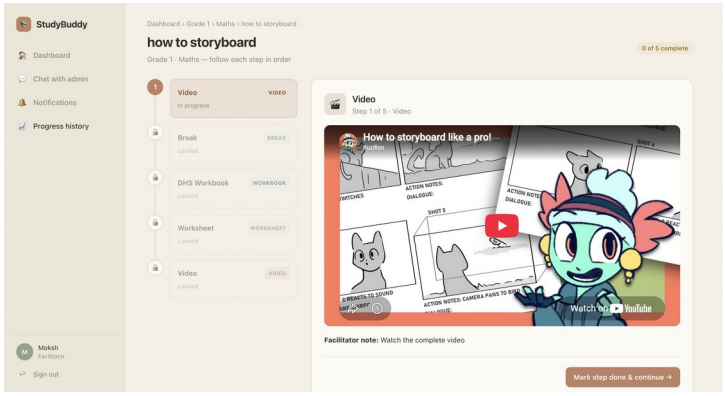


Figure 3: The block picker in the lesson authoring interface, showing all nine block types available to admins.

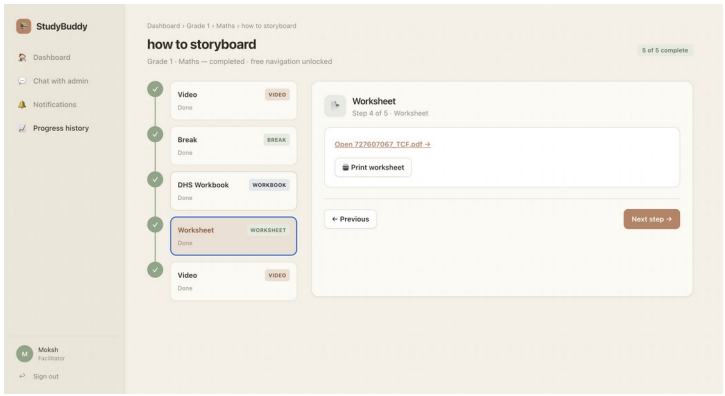
Guided delivery, with support built in. Facilitators are delivering lessons without subject expertise, so getting stuck at some point is inevitable, and there needed to be a way to reach back to the people who designed the lesson without derailing anything. That is why direct messaging between facilitators and admins is built directly into the platform, with notifications scoped to the specific conversation they belong to so reading one thread doesn't clear unrelated unread messages.

A facilitator is expected to go through a lesson once themselves before delivering it live, so they understand its flow and have whatever materials it calls for ready in advance, which is why a lesson is locked the first time a facilitator opens it, with only its first block available and each subsequent block unlocking only once the one before it is finished. Finishing a block surfaces a short prompt asking whether to move on or ask a question, putting that same messaging feature exactly where a facilitator is most likely to need it. Once every block in a lesson has been completed this way, the lesson stays fully unlocked from then on, with no further restriction on how a facilitator moves through it (Figure 4).

Built to keep growing. As more facilitators join and more classes and content get added, the system needs to keep working without becoming harder to maintain, so the code was written to be modular, easy to understand, and stable rather than quick and disposable. Classes and subjects show this in practice: they only existed through a developer-run seed script at first, but because the rest of the app already read them from the same underlying data, proper admin tools could be added later without needing any rework elsewhere. The AI worksheet generator is still on the to-do list, with a placeholder standing in for now until a real version is built. The visual design system set at the start has still been followed exactly by every feature added since. It builds cleanly for production and is live at study-buddy-uoa-project.vercel.app.



(a) Lesson opened for the first time: only the first block is unlocked.



(b) After every block is completed: fully unlocked, free navigation.

Figure 4: A lesson's guided delivery flow, before and after completion.

8. CURRENT STATUS & NEXT STEPS

All features described in Section 7 are built and tested, and the platform is live in production at study-buddy-uoa-project.vercel.app, with one facilitator account seeded as a live demonstration of the assignment and lesson-delivery flow. Priorities for the second half of the project are finalising ethics sign-off so user interviews can begin, and replacing the worksheet generator's mock logic with a real AI call. Findings from the planned interviews will directly inform prioritisation of remaining design and development work ahead of the final report.